

Day 2: 순수 Java 기반 BCrypt + JWT 인증 서버

목표: 객체지향적 설계(SRP)로 책임을 분리하고, BCrypt 암호화 및 JWT 발급을 순수 Java로 구현합니다.

1. 라이브러리 추가 (app/build.gradle)

dependencies 블록에 BCrypt와 JWT 라이브러리를 추가합니다.

```
dependencies {  
    // BCrypt  
    implementation 'org.mindrot:jbcrypt:0.4'  
  
    // JWT  
    implementation 'io.jsonwebtoken:jjwt-api:0.11.5'  
    runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.11.5'  
    runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.11.5'  
}
```

저장 후 Gradle 의존성을 동기화합니다.

2. 핵심 클래스 분리 작성 (com.example.auth 패키지)

1) User.java (도메인)

```
package com.example.auth;  
  
import org.mindrot.jbcrypt.BCrypt;  
  
public class User {  
    private final String id;  
    private final String hashedPw;  
  
    private User(String id, String hashedPw) {  
        this.id = id;  
        this.hashedPw = hashedPw;  
    }  
  
    public static User of(String id, String rawPw) {  
        String hash = BCrypt.hashpw(rawPw, BCrypt.gensalt(12));  
        return new User(id, hash);  
    }  
  
    public boolean matchPw(String rawPw) {  
        return BCrypt.checkpw(rawPw, this.hashedPw);  
    }  
}
```

```
    public String getId() { return id; }  
}
```

2) TokenProvider.java (JWT 발급/검증)

```
package com.example.auth;  
  
import io.jsonwebtoken.Jwts;  
import io.jsonwebtoken.security.Keys;  
import java.util.Date;  
import java.security.Key;  
  
public class TokenProvider {  
    private final String SECRET = "day2-secret-key-must-be-at-least-32-  
chars!!";  
    private final Key key = Keys.hmacShaKeyFor(SECRET.getBytes());  
  
    public String generate(String userId) {  
        return Jwts.builder()  
            .setSubject(userId)  
            .setExpiration(new Date(System.currentTimeMillis() +  
1800000)) // 30분  
            .signWith(key)  
            .compact();  
    }  
}
```

3) AuthService.java (비즈니스 로직)

```
package com.example.auth;  
  
import java.util.HashMap;  
import java.util.Map;  
  
public class AuthService {  
    private final TokenProvider tokenProvider;  
    private final Map<String, User> store = new HashMap<>();  
  
    public AuthService(TokenProvider tokenProvider) {  
        this.tokenProvider = tokenProvider;  
    }  
  
    public void register(String id, String pw) {  
        if (store.containsKey(id)) throw new RuntimeException("이미 존재하는  
ID");  
        store.put(id, User.of(id, pw));  
    }  
  
    public String login(String id, String pw) {
```

```
        User user = store.get(id);
        if (user == null || !user.matchPw(pw)) {
            throw new RuntimeException("인증 실패");
        }
        return tokenProvider.generate(id);
    }
}
```

4) **App.java** (수동 DI 테스트) 기존 **App.java**를 수정합니다.

```
package com.example.auth;

public class App {
    public static void main(String[] args) {
        TokenProvider tp = new TokenProvider();
        AuthService service = new AuthService(tp);

        System.out.println("회원가입 진행...");
        service.register("alice", "pass1234");

        System.out.println("로그인 진행...");
        String token = service.login("alice", "pass1234");
        System.out.println("발급된 JWT: " + token);
    }
}
```

3. 실행 및 테스트

```
./gradlew run
```